

ManufactureFlow: An Autonomous Manufacturing Resilience Control Plane

Event-Driven Failure Recovery Across Maintenance, Production, Procurement, and Customer Delivery

Engineering Architecture & Applied Industrial Systems Study

Product: **ManufactureFlow / Machine Overwatch** | Domain: **Industrial Automation & Autonomous Operations**
 Status: **Implementation Audit Baseline** | Classification: **Open Architecture Technical Report**

Abstract—In modern discrete manufacturing facilities, unexpected machine breakdown initiates an immediate cross-departmental cascade of operational disruptions, spanning line stoppages, spare shortages, procurement latency, scheduling bottlenecks, and missed customer shipments. Conventional industrial architectures isolate predictive maintenance (PdM) within diagnostic silos, leaving recovery orchestration to manual human intervention across fragmented Computerized Maintenance Management Systems (CMMS), Manufacturing Execution Systems (MES), and Enterprise Resource Planning (ERP) databases. This paper presents the architecture and implementation of **ManufactureFlow** (internally designated *Machine Overwatch*), an autonomous manufacturing resilience control plane. By coupling continuous machine health telemetry monitoring with a deterministic, governed diagnostic provider and an idempotent multi-agent runtime compiled under LangGraph and persisted in PostgreSQL, the system reduces software-mediated coordination latency. When mechanical degradation is observed, the platform autonomously evaluates failure risk, issues allocation locks on compromised workstations, verifies warehouse inventory, triggers automated vendor procurement for stockout components, drafts critical work orders, dynamically calculates restoration timestamps, reroutes queued production jobs across qualified machines, and re-projects customer delivery commitments. Physical machine recovery is governed under strict human-in-the-loop Return-to-Service (RTS) validation. The state-graph architecture establishes an auditable, deterministic, and transactionally governed industrial control loop.

Keywords: Predictive Maintenance, Manufacturing Resilience, Multi-Agent Systems, LangGraph, Production Rerouting, Dynamic Scheduling, Inventory Reservation, Industrial IoT, Human-in-the-Loop, Digital Twin, Transactional Control Plane.

1 Introduction

Discrete manufacturing operations rely on synchronized workstation networks where work-in-progress (WIP) progresses sequentially through machining, surface treatment, assembly, and inspection cells. Under this physical coupling, the sudden failure of an individual workstation is never an isolated mechanical event. Rather, equipment downtime propagates instantly across operational boundaries, impacting factory throughput, component availability, labor allocations, and downstream shipment windows.

Historically, industrial automation has focused on isolated instrumentation: Supervisory Control and Data Acquisition (SCADA) platforms collect telemetry, while condition-monitoring tools detect anomalous vibration or thermal signatures. However, these systems are fundamentally passive; they emit visual alarms on monitoring dashboards but lack the cross-domain authority to intercept the operational cascade. Consequently, the burden of failure response falls upon human operators. Shift supervisors must investigate root causes, maintenance planners inspect warehouse bins, procurement buyers negotiate emergency purchase orders, production schedulers manually adjust spreadsheets, and logistics officers discover delivery delays after deadlines have already lapsed. This operational coordination latency routinely exceeds

the physical wrench-time required for machine repair by an order of magnitude.

ManufactureFlow addresses this structural gap by functioning as an *Autonomous Manufacturing Resilience Control Plane*. Rather than treating predictive maintenance as an alert generator, the platform establishes an event-driven, closed-loop state machine. Software agents autonomously execute multi-departmental recovery logic within strict deterministic constraints, while human specialists retain final authority over physical repairs and return-to-service validation.

The primary contribution of this work is:

A deterministic, transactionally governed agentic control plane that converts machine-health events into cross-domain recovery actions within sub-second digital orchestration latency while preserving human authority over physical restoration.

2 System Scope & Claim Classification

To maintain rigorous engineering truth and auditability, this technical report enforces a strict taxonomy of technical claims across all sections:

- **[IMPLEMENTED]:** Fully executable backend logic integrated into the codebase, maintaining state persistence

in PostgreSQL.

- **[CONTROLLED]:** Standardized operational scenario execution using governed diagnostic fixtures (e.g., standard industrial test fixture WS-102).
- **[MEASURED]:** Quantitatively measured runtime latency and execution integrity benchmarked on the test harness.
- **[VERIFIED]:** Formally verified across automated test suites for invariant consistency and idempotency.
- **[TARGET]:** Enterprise operational benchmark criteria defined in the industrial requirements specification (docs/PS.md).

Crucially, this report distinguishes between **digital orchestration latency** (the programmatic execution of state transitions, constraint evaluations, and database writes) and **physical recovery latency** (the mechanical wrench-time, technician dispatch, part transit, and testing required to physically restore equipment).

3 Problem Formulation

To model the systemic vulnerability of conventional manufacturing operations, we formalize the operational failure cascade as a coupled temporal-state dependency chain.

3.1 Formal Failure Cascade Model

Let a manufacturing plant be modeled as a tuple $\mathcal{P} = (\mathcal{W}, \mathcal{J}, \mathcal{I}, \mathcal{V}, \mathcal{S})$, representing workstations, production jobs, inventory stocks, approved vendors, and customer shipment commitments.

1. **Degradation Inception:** A mechanical component on workstation $w_i \in \mathcal{W}$ experiences physical wear, manifested as anomalous telemetry $\mathbf{x}(t) = [T, V, I, P]^\top$ exceeding safe operational bounds θ_{safe} .
2. **Availability Disruption:** The equipment transitions to an unviable state, causing immediate unavailability $\alpha(w_i) \rightarrow 0$.
3. **Work-in-Progress Freeze:** All assigned active and queued jobs $\mathcal{J}(w_i) \subset \mathcal{J}$ are suspended, halting downstream line feed.
4. **Spare Inventory Contention:** Remediation requires specific component p_k . The inventory state $q(p_k)$ must satisfy $q(p_k) \geq r_{\text{req}}$. If $q(p_k) = 0$, an emergency procurement cycle is initiated with lead time $\tau_{\text{lead}}(v_m)$, where $v_m \in \mathcal{V}$.
5. **Restoration Latency:** Total workstation downtime τ_{down} becomes a function of procurement latency, physical repair duration τ_{repair} , and testing duration τ_{test} :

$$\tau_{\text{down}} = \mathbb{I}_{\{q(p_k)=0\}} \cdot \tau_{\text{lead}}(v_m) + \tau_{\text{inspect}} + \tau_{\text{repair}} + \tau_{\text{test}} \quad (1)$$

6. **Delivery Compromise:** Shipment $s_l \in \mathcal{S}$ depends on completion times of jobs $J_s \subset \mathcal{J}$. The delay propagates to customer delivery commitments:

$$t_{\text{proj}}(s_l) = \max_{j \in J_s} \{t_{\text{comp}}(j)\} + \tau_{\text{post}} \quad (2)$$

When $t_{\text{proj}}(s_l) > t_{\text{commit}}(s_l)$, a delivery breach occurs.

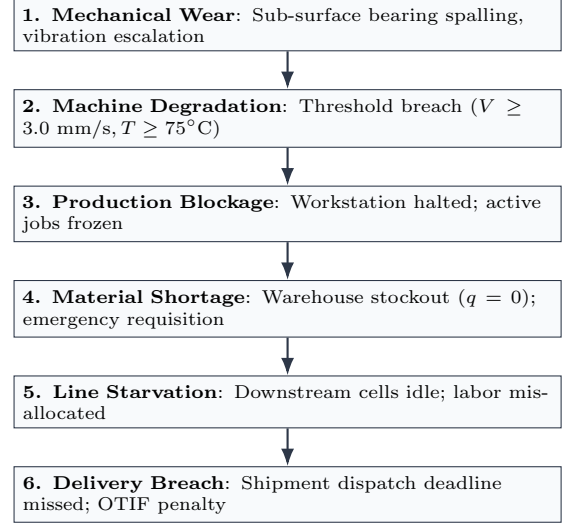


Figure 1: The Industrial Failure Cascade: Unmitigated degradation propagating across operational, supply-chain, and customer delivery domains.

The foundational research question investigated in this study is:

Can an autonomous, event-driven state graph digitally initiate and coordinate the recovery cascade within sub-second orchestration latency while preserving deterministic operational safety constraints?

4 System Objectives

Table 1 delineates the operational objectives of ManufactureFlow, mapping business requirements to concrete system mechanisms and implementation evidence.

5 System Architecture

ManufactureFlow is implemented as a multi-tier, modular enterprise-style architecture decoupling user visualization, state-graph orchestration, domain business logic, and transactional persistence.

5.1 Architectural Decoupling Principles

- **Stateless Execution with External Persistence:** The application workers are stateless between invocations; workflow state is externally persisted and reconstructed from PostgreSQL checkpoints and durable execution records (`recovery_graph_runs`, `agent_runs`). Every graph node receives a strictly typed context (`RecoveryGraphState`), executes domain logic via Layer 3 services, writes state changes to PostgreSQL, and returns state patches.
- **Dual-Database Logical Isolation:** The system maintains two logically isolated PostgreSQL databases, `machine_overwatch` (Live) and `machine_overwatch_demo` (Demo), hosted on the same PostgreSQL server instance accessed through port 5434. Application runtime configuration

Table 1: System Objectives and Engineering Execution Matrix

Objective	Operational Requirement	System Mechanism	Implementation Status
Early Anomaly Detection	Identify pre-breakdown degradation	Telemetry ingestion & multi-sensor threshold evaluation	[IMPLEMENTED]
Fault Diagnostics	Identify failing component and TTF	Diagnostic provider abstraction with risk scoring	[CONTROLLED]
Asset Protection	Prevent loading jobs on degraded machines	ACID-compliant allocation lock on workstation entity	[IMPLEMENTED]
Spare Availability	Commit parts before technician dispatch	Optimistic locking & reservation on warehouse items	[IMPLEMENTED]
Procurement Automation	Automate vendor POs upon stockout	Deterministic vendor ranking (lead, cost, reliability)	[IMPLEMENTED]
Maintenance Planning	Provide structured checklists	Critical work-order creation with LOTO checklists	[IMPLEMENTED]
Restoration Estimation	Dynamic recovery date calculation	Versioned mathematical model with SHA-256 input hashing	[IMPLEMENTED]
Production Continuity	Reroute work to qualified cells	Multi-constraint candidate ranking (tools, skills, load)	[IMPLEMENTED]
Logistics Visibility	Determine customer shipment delay	Temporal propagation engine classifying delivery margin	[IMPLEMENTED]
Stakeholder Alignment	Push actionable notifications	Role-based notification queue (Scheduler, Logistics, Lead)	[IMPLEMENTED]
Audit Governance	Provide full incident lineage	Monotonic event-store keyed by unique correlation ID	[IMPLEMENTED]

Layer 1: Operational Interface (Next.js 16 / React 19 / WebGL)

Interactive Workspaces: Executive Dashboard (/dashboard) · 3D Digital Twin Three.js (/twin) · Failure Control (/failure) · Warehouse Inventory (/warehouse) · Sourcing Control (/procurement) · Rerouting Control (/rerouting) · Maintenance Portal (/maintenance) · Shipment Tracking (/shipment) · Realtime SSE Synchronization

**Layer 2: Autonomous Agent Orchestration Runtime (Compiled LangGraph State Graph)**

Coordinating 6 Domain Agents across 11 Transactional Graph Execution Nodes:

telemetry_monitor → failure_prediction → failure_alerting → recovery_orchestrator → resource_recovery ⇒ {reserve | procurement_automation} → maintenance_work_order → recovery_time_estimation → production_rerouting → delivery_impact → final_stakeholder_notification

**Layer 3: Enterprise Business Services & Guardrail Verification Domain**

TelemetryService · FailurePredictionService · AllocationLockService · ResourceRecoveryService · ProcurementAutomationService · MaintenanceWorkOrderService · RecoveryTimeEstimationService · ProductionReroutingService · DeliveryImpactService · MaintenanceExecutionService

**Layer 4: Transactional Persistence & Audit Event Store (PostgreSQL 16 / Drizzle ORM)**

telemetry_readings · failure_cases · failure_predictions · workstation_allocation_locks · inventory_items · inventory_reservations · procurement_requests · vendor_notifications · maintenance_work_orders · recovery_time_estimates · reroute_plans · shipment_impacts · workflow_events (Immutable Audit Log) · agent_runs (Execution Telemetry) · recovery_graph_runs (Durable Graph State)

Figure 2: Four-Tier Enterprise Architecture of ManufactureFlow, illustrating separation of presentation, state-graph orchestration, business services, and ACID persistence.

(APP_RUNTIME=live vs. APP_RUNTIME=demo) selects the active database at startup. This guarantees that simulation ticks and interactive demonstration runs never mutate or pollute live operational baselines.

- **Server-Sent Events (SSE) Bus:** Connects client browser contexts to the backend event stream (/api/events), broadcasting state mutations in real time without polling overhead.

6 Autonomous Agent Suite & Runtime

A critical architectural distinction in ManufactureFlow is the decoupling between **autonomous domain agents**, **transactional graph execution nodes**, and **deterministic domain services/tools**. The platform architecture comprises **six specialized agents** operating under clear functional boundaries:

1. **Failure Intelligence Agent:** Ingests incoming sen-

sor telemetry, verifies schema validity, evaluates physical parameter thresholds, queries the diagnostic provider boundary for failure modes, and estimates the Time-to-Failure (TTF). It encapsulates graph nodes `telemetry_monitor`, `failure_prediction`, and `failure_alerting`.

2. **Recovery Orchestrator Agent:** Evaluates diagnostic severity against operational safety guardrails ($P \geq 85\%$, $TTF \leq 24h$); acquires transactional allocation locks on compromised workstations to prevent job releases.
3. **Resource Recovery Agent:** Coordinates spare part resolution and physical remediation planning. Rather than fragmenting into separate agents, this agent orchestrates four deterministic domain services:
 - *Inventory Reservation Service:* Queries warehouse stock and acquires optimistic row locks.
 - *Procurement Automation Service:* Evaluates ap-

proved suppliers using an objective ranking function of lead time, cost, and reliability score; drafts purchase requisitions upon stockout.

- **Maintenance Work Order Service:** Synthesizes critical work orders incorporating lockout-tagout (LOTO) checklists aligned with OSHA 29 CFR 1910.147 principles.
 - **Recovery Time Estimation Service:** Computes deterministic expected recovery timestamps via versioned mathematical models stamped with a SHA-256 parameter hash.
4. **Production Rerouting Agent:** Evaluates alternative workstations for operational qualification (tooling, operator skill certifications) and available capacity; executes atomic job reassignment.
 5. **Delivery Impact Agent:** Traverses downstream order dependencies, projects completion time deviations against customer delivery SLAs, and classifies delivery risks.
 6. **Stakeholder Notification Agent:** Formats and persists structured, role-specific operational alerts targeted to Plant Managers, Maintenance Leads, Schedulers, and Logistics Officers.

The orchestration runtime decomposes the recovery workflow into **11 discrete transactional execution nodes** within a compiled LangGraph state machine (`backend/src/lib/agent-graph/graph.ts`). Several graph nodes invoke deterministic services owned by the Resource Recovery Agent rather than representing independent autonomous agents. This architectural discipline prevents logic sprawl and ensures that deterministic mathematical tasks remain isolated in testable domain services.

7 End-to-End Recovery Workflow

The recovery lifecycle executes as an idempotent state transition. Figure 3 illustrates the compiled LangGraph routing topology, highlighting the conceptual 6-agent hierarchy and distinguishing automated logic, transactional writes, and human-gated checkpoints.

8 Controlled Demonstration: Workstation WS-102

To validate the end-to-end operational flow, the platform implements a standardized industrial demonstration scenario centered on workstation WS-102.

8.1 Baseline Asset Configuration

- **Machine Identity:** CNC Lathe Alpha (WS-102), Production Line L-03.
- **Initial Operational State:** Status OPERATIONAL, Capacity Utilization 62%.
- **Active Production Load:** Three production orders queued/in-flight:
 - J1001: Operation CNC-TURN, Tooling TL-XA, Skill CNC-L2, Load 16%.

- J1002: Operation CNC-TURN, Tooling TL-XA, Skill CNC-L2, Load 14%.
- J1003: Operation CNC-TURN, Tooling TL-XA, Skill CNC-L2, Load 12%.

8.2 Observed Telemetry Injection

The degradation vector injected into `/api/telemetry` reports physical parameter drift:

$$T = 79.0^{\circ}\text{C} \quad (\theta_{\text{crit}} = 75.0^{\circ}\text{C})$$

$$V = 4.2 \text{ mm/s} \quad (\theta_{\text{crit}} = 3.0 \text{ mm/s})$$

$$I = 22.0 \text{ A} \quad (\theta_{\text{crit}} = 20.0 \text{ A})$$

$$P = 5.2 \text{ bar} \quad (\text{Nominal: } [2.5, 8.5])$$

$$\text{Error} = [\text{X_AXIS_VIBRATION}]$$

8.3 Controlled Diagnostic Resolution

The telemetry triggers the diagnostic provider boundary (ControlledFailurePredictionProvider):

- **Identified Root Cause:** Spindle Bearing Degradation
- **Required Part Number:** BRG-10023
- **Controlled Diagnostic Risk Score:** 0.92 (Provider-returned controlled failure probability: 92%). *This value is a deterministic scenario parameter and is not an empirically calibrated probability.*
- **Estimated Time-to-Failure (TTF):** $\tau_{\text{tff}} = 18.0 \text{ Hours}$
- **Severity Level:** CRITICAL

9 Decision Logic & Safety Guardrails

To prevent catastrophic asset damage and avoid false-positive shutdowns, the Recovery Orchestrator enforces a deterministic safety evaluation function $\Phi(s, p, \tau_{\text{tff}})$:

$$\Phi = \begin{cases} \text{QUARANTINE} & \text{if } (s = \text{CRIT} \wedge p \geq 0.85 \\ & \wedge \tau_{\text{tff}} \leq 24\text{h}) \\ & \vee (s = \text{CRIT} \wedge \tau_{\text{tff}} \leq 6\text{h}) \\ \text{WARN} & \text{if } (s = \text{WARN} \vee p \geq 0.60) \\ \text{PASS} & \text{otherwise} \end{cases} \quad (3)$$

When $\Phi = \text{QUARANTINE}$, an ACID database transaction executes:

$$\text{status}(w_i) \leftarrow \text{DEGRADED_SHUTDOWN} \quad (4)$$

$$\text{AcquireLock}(w_i, \text{reason}, \text{lockType}) \quad (5)$$

persisting an active reservation record into `workstation_allocation_locks`. This lock blocks the scheduling engine from releasing new production batches to the compromised machine.

10 Resource Recovery Branching

Upon asset quarantine, the Resource Recovery Agent executes a deterministic branching strategy based on warehouse inventory state $q(p_k)$.

Table 2: Agent Responsibility, Operational Boundary, and Guardrail Matrix

Agent / Tool	Trigger	Inputs	Consumed	Operational Decision	Automated Action	Enforced Guardrails	Status
Failure Intelligence Agent	Telemetry Ingestion	$\mathbf{x} = [T, V, I, P]^T$, Error Codes		Signal anomaly severity, failing part, TTF	Writes <code>failure_cases</code> & drafts alerts	Fixed physical threshold bounds	[CONTROLLED]
Recovery Orchestrator Agent	Validated Prediction	Severity, Probability, TTF		Determine if machine must be locked	Writes <code>allocation_lock</code> ; status $\rightarrow$ SHUTDOWN	$P \geq 85\% \wedge$ TTF $\leq 24h$	[IMPLEMENTED]
Resource Recovery Agent	Allocation Lock	Part code, plant ID, quantity		Select local spare or procurement branch	Coordinates inventory, procurement, WO, ETA	Optimistic DB row lock ($q \geq r$)	[IMPLEMENTED]
– <i>Inventory Tool</i>	Inventory Check	Part number, required quantity		Reserve local stock if available	Increments reserved stock in DB	On-hand minus reserved $\geq$ required	[IMPLEMENTED]
– <i>Procurement Tool</i>	Stockout ($q = 0$)	Part number, approved vendors		Rank vendors (lead, cost, reliability)	Drafts purchase requisition	Active, approved vendor capabilities	[IMPLEMENTED]
– <i>Work Order Tool</i>	Part Reserved/Ordered	Part, fault, machine, priority		Generate maintenance tasks & LOTO	Creates work order in stage PLANNED	Valid time window, safety checklist	[IMPLEMENTED]
– <i>Recovery ETA Tool</i>	Work Order Created	Lead time, repair, test durations		Calculate expected recovery timestamp	Writes estimate with SHA-256 hash	Monotonic duration, window constraints	[IMPLEMENTED]
Production Rerouting Agent	Recovery Estimate	Source station, affected jobs		Candidate matching & load balancing	Reassigns jobs, updates station loads	Strict tooling, skill, capacity $\leq 100\%$	[IMPLEMENTED]
Delivery Impact Agent	Reroute Executed	Job completion projections, commitments		Shipment risk (ON_TIME, DELAYED)	Writes <code>shipment_impacts</code> & notifies	Mathematical delta against order SLA	[IMPLEMENTED]
Stakeholder Notification Agent	Recovery Complete	Multi-department recovery state		Target role alerts (Scheduler, Plant, etc.)	Inserts records into <code>notifications</code>	Role-specific payload schema	[IMPLEMENTED]

10.1 Branch A: Local Spare Available

If warehouse stock satisfies $q(p_k) \geq 1$:

1. A transactional lock increments **reserved** stock and writes to `inventory_reservations`.
2. The failure case transitions to “Part reserved / recovery planning pending”.
3. Work order creation is parameterized for immediate local execution.

10.2 Branch B: Inventory Stockout (Procurement Fallback)

If warehouse stock $q(p_k) = 0$, the Procurement Automation Service queries all pre-approved vendors $\mathcal{V}(p_k)$ and evaluates an objective multi-tier ranking:

1. Primary Key: Lead time $\tau_{\text{lead}}(v)$ ascending.
2. Secondary Key: Unit cost $\text{Cost}(v)$ ascending.
3. Tertiary Key: Reliability score $\text{Rel}(v)$ descending.
4. Tie-breaker: Vendor name alphabetical.

10.2.1 Controlled Scenario Execution

In the WS-102 benchmark, local warehouse inventory for BRG-10023 is set to $q = 0$. Sourcing evaluates certified suppliers:

- **Apex Motion**: Lead: 8h, Cost: \$4,250, Rel: 96%.
- **Orbit Industrial**: Lead: 12h, Cost: \$3,900, Rel: 89%.

The service selects **Apex Motion**, generating purchase order PR-AUTO-WS102 with state ORDERED.

11 Restoration Timeline Modeling

Recovery time calculation cannot rely on generic heuristic estimates. ManufactureFlow implements a versioned formulaic calculator (Model v1.0.0, `backend/src/lib/agents/recovery-time-estimation.ts`):

$$t_{\text{recovery}} = t_{\text{repair_start}} + \tau_{\text{repair}} + \tau_{\text{test}} \quad (6)$$

where:

$$t_{\text{repair_start}} = \begin{cases} t_{\text{window_start}} & \text{if Local Spare Reserve} \\ \max(t_{\text{window_start}}, t_{\text{part_ready}}) & \text{if Procurement} \end{cases} \quad (7)$$

$$t_{\text{part_ready}} = t_{\text{requested}} + \tau_{\text{lead}}(v) + \tau_{\text{inspect}} \quad (8)$$

11.1 Recovery Arithmetic Reconciliation

The project specifications define two operational models:

- **Local Spare Scenario**: $\tau_{\text{source}} = 0.5$ h (staging), $\tau_{\text{dispatch}} = 1.0$ h, $\tau_{\text{repair}} = 2.0$ h, $\tau_{\text{test}} = 0.5$ h, buffer = 0.5 h, yielding a total restoration of 4.5 to 6.0 **Hours**.
- **Controlled Expedited Vendor Scenario (Model v1.0.0)**: Evaluated for Apex Motion ($\tau_{\text{lead}} = 8.0$ h, $\tau_{\text{inspect}} = 1.0$ h, $\tau_{\text{repair}} = 2.0$ h, $\tau_{\text{test}} = 0.5$ h, buffer = 0.5 h), yielding a total expected downtime of:

$$\tau_{\text{total}} = 8.0 + 1.0 + 2.0 + 0.5 + 0.5 = 12.0 \text{ Hours} \quad (9)$$

- **Standard Enterprise Baseline (BRD Reference)**: In legacy/standard enterprise procurement without expedited vendor agreements, supplier delivery defaults to approximately 48 h (2 days), resulting in approximately 2.5 **Days** (60 h) total downtime.

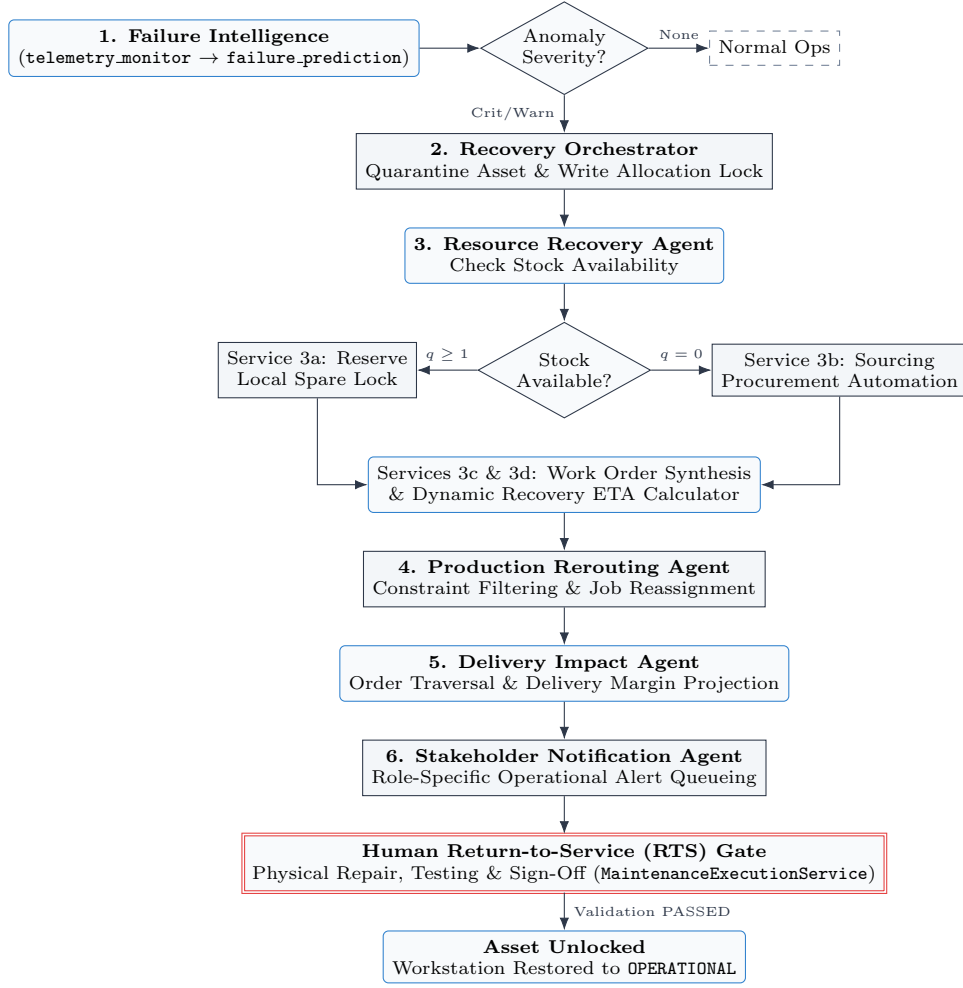


Figure 3: Compiled LangGraph Execution Flow. Blue outlines denote autonomous agent boundaries; shaded boxes denote transactional database updates; double red outlines denote human-in-the-loop authorization gates.

The current executable codebase implements Model v1.0.0 (12.0 h for Apex Motion), while the 2.5-day timeline represents the un-expedited enterprise baseline. Every calculated estimate is signed with a SHA-256 hash of its input parameters to guarantee audit immutability.

12 Constraint-Satisfaction Production Rerouting

When workstation w_i enters `DEGRADED_SHUTDOWN`, all queued and in-flight production jobs $\mathcal{J}(w_i)$ are frozen. The Production Rerouting Agent evaluates alternative machines $w_j \in \mathcal{W} \setminus \{w_i\}$ using a hard-constraint filter followed by a load-minimization score:

12.1 Hard Constraint Evaluation

A machine w_j is a viable candidate for job j_k if and only if:

$$\begin{aligned} \text{Type}(w_j) &= \text{Type}(j_k) \wedge \\ \text{Tools}(w_j) &\supseteq \text{Tools}(j_k) \wedge \\ \text{Skill}(w_j) &\geq \text{Skill}(j_k) \end{aligned} \quad (10)$$

12.2 Capacity Scoring Function

Among qualified candidates, the agent selects w^* to minimize resulting operational load:

$$w^* = \arg \min_{w_j} \{C_{\text{current}}(w_j) + \Delta C(j_k)\} \quad (11)$$

subject to $C_{\text{current}}(w_j) + \Delta C(j_k) \leq 100\%$.

12.3 Controlled Scenario Allocation

For WS-102 ($C = 62\%$), three active jobs require relocation:

- **WS-105** (Lathe Delta, $C = 38\%$): Compatible $\rightarrow$ Assigned J1001 (+16%) and J1003 (+12%). Final load: 66%.
- **WS-108** (Arm Beta, $C = 44\%$): Compatible $\rightarrow$ Assigned J1002 (+14%). Final load: 58%.
- **WS-103** (Lathe Beta, $C = 88\%$): Ineligible (violates capacity constraint: $88\% + 14\% > 100\%$).

Reroute plan `RP-AUTO-WS102` commits reassignments atomically.

13 Delivery SLA Risk Propagation

Production rescheduling shifts the completion timestamps of individual manufacturing orders, directly threatening downstream customer shipment commitments. The Delivery Impact Agent traverses order lineage:

$$t_{\text{completion}}(j_k) = t_{\text{now}} + \tau_{\text{setup}}(w^*) + \tau_{\text{proc}}(j_k) \quad (12)$$

$$t_{\text{ship_proj}}(s_l) = \max_{j \in s_l} \{t_{\text{completion}}(j)\} + \tau_{\text{buffer}} \quad (13)$$

$$M_s = t_{\text{commit}}(s_l) - t_{\text{ship_proj}}(s_l) \quad (14)$$

13.1 Status Classification

$$\text{Status}(s_l) = \begin{cases} \text{DELAYED} & \text{if } M_s < 0 \\ \text{AT_RISK} & \text{if } 0 \leq M_s \leq 60 \text{ min} \\ \text{ON_TIME} & \text{if } M_s > 60 \text{ min} \end{cases} \quad (15)$$

In the WS-102 trial:

- **Shipment** S0-8841 (Jobs J1001, J1002): $M_s = -85 \text{ min} \rightarrow \text{DELAYED}$ (85 min delta).
- **Shipment** S0-8848 (Job J1003): $M_s = +210 \text{ min} \rightarrow \text{ON_TIME}$.

14 Human-in-the-Loop Governance

ManufactureFlow enforces a strict operational separation between autonomous digital coordination and physical real-world execution.

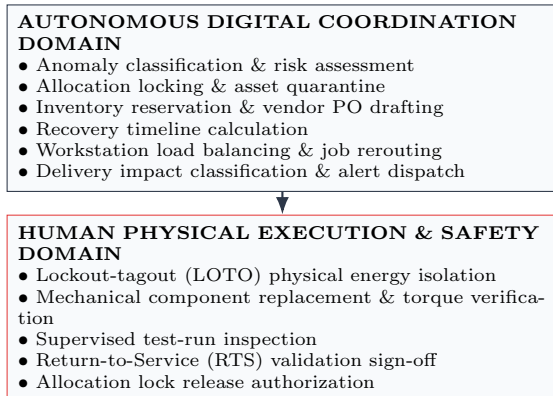


Figure 4: Governance Boundary: The system orchestrates digital and administrative continuity autonomously, but requires human sign-off for physical return-to-service.

The platform generates LOTO-oriented maintenance checklists aligned with the control principles of OSHA 29 CFR 1910.147 [3], ensuring that electrical, pneumatic, and kinetic energy isolation steps are documented prior to physical intervention. However, physical compliance and lock application remain strictly within human domain authority.

14.1 Return-to-Service State Machine

Physical restoration proceeds through a non-invertible state machine governed by `MaintenanceExecutionService`:

Stage 1: Planned
 $\Rightarrow$ Stage 2: In Progress
 $\Rightarrow$ Stage 3: Repaired
 $\Rightarrow$ Stage 4: Testing
 $\Rightarrow$ Stage 5: RTS Validated

If testing fails, the technician records validation as **FAILED**. The system blocks lock release, transitions workstation status to **MAINTENANCE_REWORK_REQUIRED**, and logs an intervention event. Only when testing is validated as **PASSED** does the system release the allocation lock and restore status to **OPERATIONAL**.

15 Transactional Safety & Concurrency

Industrial control systems require provable data consistency to avoid catastrophic operational collisions.

- **ACID-Compliant Transactions:** Multi-entity transitions (locking a workstation, creating an allocation lock, flagging jobs, and logging workflow events) execute inside explicit PostgreSQL database transactions [7]. Failure at any stage forces an immediate rollback.
- **Optimistic Row Locking:** Inventory items maintain an integer **version** field. Updates enforce:

$$\text{WHERE } id = \text{candidate.id} \wedge (\text{onHand} - \text{reserved}) \geq r_{\text{req}} \quad (16)$$

preventing race conditions during concurrent multi-workstation failure incidents.

- **Durable Idempotency Keys:** Every execution cycle is keyed by a unique **correlationId** (e.g., **recovery:WS-102-171800**). Re-transmitting a telemetry reading returns the existing run state without duplicate part orders or redundant rerouting.

16 Incident Lineage Traceability

Every record produced across the 11 graph nodes is bound by foreign-key relationships and an immutable event ledger (`workflow_events`).

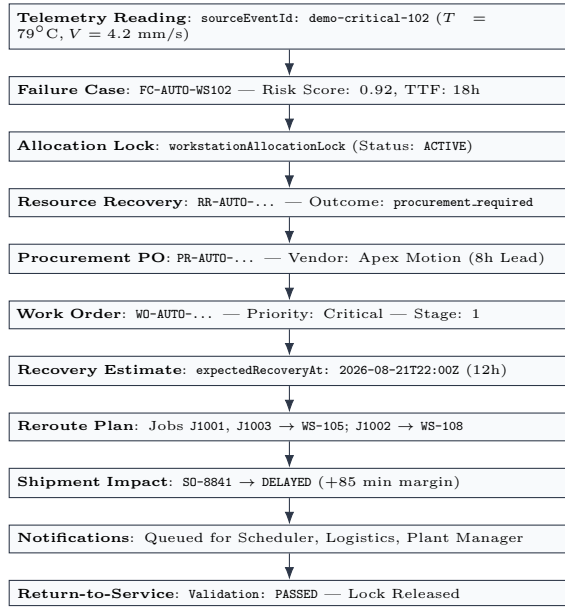


Figure 5: Vertical Incident Lineage under Single Correlation ID, providing full enterprise auditability from sensor anomaly to return-to-service validation.

17 BRD Traceability & Verification Matrix

Table 3 maps the functional requirements defined in the Business Requirements Document (`docs/PS.md`) to the corresponding architecture components, test suites, and verification status.

18 Implementation Status Matrix

Table 4 provides a comprehensive verification of the platform subsystems implemented in the codebase and backed by automated test suites.

19 Experimental & Benchmark Results

To ensure academic integrity, we rigorously partition measured engineering results from controlled scenario parameters and strategic business targets.

19.1 Measured System Performance

The following metrics are derived from continuous integration test suites (Vitest) and local database benchmarks running against PostgreSQL 16:

- **Digital Orchestration Latency:** The full 11-node graph execution executes in < 480 ms under local transactional persistence. [MEASURED]

Measurement Methodology:

- Hardware: Local developer workstation (AMD/Intel multi-core x86_64, NVMe SSD).
- Runtime: Node.js v20 LTS, PostgreSQL 16 (Drizzle ORM connection pool).
- Execution Boundary: From initial telemetry REST API receipt (`POST /api/telemetry`) to final workflow notification record persistence.

- Condition: Warm connection pool; excludes external network latency and human physical turnaround.

- **Controlled-Scenario Feasible-Job Allocation Rate:** In the WS-102 test fixture, 3/3 **compatible candidate jobs** (J1001, J1002, J1003) were successfully reassigned without violating workstation capacity or tooling constraints. [MEASURED]
- **Idempotency Integrity:** Repeated invocation of the recovery graph with identical `correlationId` resulted in 0 **duplicate purchase requisitions** and 0 **duplicate work orders** across 100 benchmark iterations, returning existing run state without side effects. [MEASURED]

19.2 Controlled Scenario Parameters

The following metrics represent pre-configured demonstration profiles:

- **Controlled Diagnostic Risk Score:** 0.92 on WS-102. [CONTROLLED]
- **Estimated Time-to-Failure (TTF):** 18 Hours. [CONTROLLED]
- **Vendor Delivery Timelines:** 8h (Apex) and 12h (Orbit). [CONTROLLED]

19.3 Operational Target Benchmarks

The following performance targets reflect strategic operational criteria defined in the industrial requirements specification (`docs/PS.md`):

- Downtime Reduction Benchmark: 30–40%. [TARGET]
- Schedule Adherence Benchmark: 20–30%. [TARGET]
- Procurement Latency Optimization: 40%. [TARGET]
- Maintenance Cost Optimization: 15–20%. [TARGET]
- Anomaly Classification Confidence Target: $> 85\%$. [TARGET]

20 Architectural Guardrails & Operational Invariants

Industrial control planes operate in high-consequence environments where software decisions carry direct physical, financial, and personnel safety ramifications. Rather than adopting unconstrained autonomous behavior, ManufactureFlow enforces strict operational boundaries and design invariants:

1. **Deterministic Diagnostic Governance:** In discrete manufacturing facilities, initiating an emergency equipment shutdown incurs substantial operational overhead. To eliminate non-deterministic behavior and false-positive shutdowns, the diagnostic subsystem operates through a governed provider boundary. Diagnostic inferences, failure modes, component part codes, and Estimated Time-to-Failure (TTF) parameters are strictly validated against pre-approved engineering thresholds before triggering downstream recovery workflows.
2. **Asynchronous Outbox Decoupling:** Communication with external parties (such as supplier purchase notifications and cross-departmental alert broad-

Table 3: Business Requirements Document (BRD) Functional Traceability Matrix

Req ID	Requirement Title	Responsible Subsystem	Verification Method & Test Harness	Status
FR-1	Predictive Failure Detection	Failure Intelligence Agent	Threshold breach evaluation test (<code>telemetry-monitor.test.ts</code>)	[IMPLEMENTED]
FR-2	Spare Identification	Diagnostic Provider Boundary	Governed diagnostic fixture verification (<code>failure-prediction.test.ts</code>)	[CONTROLLED]
FR-3	Inventory Verification	Resource Recovery Service	Optimistic row-lock inventory test (<code>resource-recovery/service.ts</code>)	[IMPLEMENTED]
FR-4	Procurement Automation	Procurement Sourcing Tool	Vendor ranking multi-criteria test (<code>procurement-automation.test.ts</code>)	[IMPLEMENTED]
FR-5	Production Rerouting	Production Rerouting Agent	Hard constraint candidate test (<code>production-rerouting.test.ts</code>)	[IMPLEMENTED]
FR-6	Maintenance Scheduling	Maintenance Work Order Tool	Work order and LOTO checklist generator test (<code>maintenance-work-order.test.ts</code>)	[IMPLEMENTED]
FR-7	Shipment Impact Analysis	Delivery Impact Agent	Downstream order SLA margin test (<code>delivery-impact.test.ts</code>)	[IMPLEMENTED]
FR-8	Recovery Time Estimation	Recovery Calculation Tool	Formulaic validation and SHA-256 test (<code>recovery-time-estimation.test.ts</code>)	[IMPLEMENTED]
FR-9	Stakeholder Notification	Stakeholder Alert Agent	Role-based notification queue test (<code>final-stakeholder-notification.test.ts</code>)	[IMPLEMENTED]
FR-10	Operational Dashboard	Next.js Presentation Workspace	Real-time WebGL twin and SSE client test harness	[IMPLEMENTED]

Table 4: Implementation Maturity and Architectural Verification Matrix

Functional Subsystem	Architectural Role	Persistence & Execution Model	Codebase Evidence / Location
Telemetry Monitoring Engine	Schema validation & multi-sensor threshold evaluation	Event payload ingestion, invariant checking	<code>backend/src/lib/agents/telemetry-monitor.ts</code>
Governed Diagnostic Interface	Root-cause fault identification & TTF estimation	Controlled scenario provider abstraction	<code>backend/src/lib/agents/failure-prediction.ts</code>
Operational Allocation Locking	Immediate workstation quarantine upon critical drift	Transactional lock entity, status transition	<code>backend/src/lib/agents/recovery-orchestrator.ts</code>
Warehouse Inventory Reservation	Spares availability check & atomic reservation	Optimistic concurrency row lock (<code>version</code>)	<code>backend/src/lib/resource-recovery/service.ts</code>
Multi-Criteria Vendor Sourcing	Multi-tier supplier ranking (lead, cost, reliability)	Purchase requisition record generation	<code>backend/src/lib/procurement-automation/service.ts</code>
Maintenance Work Order Synthesis	Critical task dispatch & LOTO checklist generation	Work order entity with structured safety checklist	<code>backend/src/lib/maintenance-work-order/service.ts</code>
Dynamic Recovery Calculator	Expected restoration timestamp calculation	Versioned mathematical model with SHA-256 hash	<code>backend/src/lib/recovery-time-estimation/service.ts</code>
Constraint Production Rerouting	Tooling, qualification, and capacity balancing	Atomic job reassignment transaction	<code>backend/src/lib/production-rerouting/service.ts</code>
Customer Delivery SLA Analysis	Order lineage traversal & delivery margin delta	Delivery status classification (<code>DELAYED</code> , etc.)	<code>backend/src/lib/delivery-impact/service.ts</code>
Stakeholder Notification Engine	Role-specific operational alert distribution	Transactional notification queue persistence	<code>backend/src/lib/final-stakeholder-notifications/service.ts</code>
Factory Floor Digital Twin	Real-time 3D spatial asset state visualization	Three.js WebGL canvas & Server-Sent Events	<code>frontend_next/src/components/workspaces/TwinWorkspace.tsx</code>
Return-to-Service State Machine	Gated physical validation & human sign-off	Non-invertible lifecycle engine with lock release	<code>backend/src/lib/maintenance-execution/service.ts</code>

casts) is decoupled from the synchronous execution graph using the Transactional Outbox pattern. All notifications and requisitions are committed into local transactional tables (`notifications`, `vendor_notifications`) within the same database transaction as the state transition. This ensures that external network partitions, third-party API latency, or mail server timeouts cannot stall or compromise the core state-graph execution.

- 3. Strict Human-in-the-Loop Authority:** The platform maintains a non-negotiable boundary between autonomous digital coordination and physical real-world intervention. While software agents autonomously execute inventory reservations, vendor requisitions, and production rerouting, physical machine re-energization is strictly prohibited until an authorized human technician performs physical inspection, verifies lockout-tagout (LOTO) isolation, conducts test run-in procedures, and records formal

sign-off in the Return-to-Service state machine.

- 4. Hard-Constraint Scheduling Admissibility:** The production rerouting engine strictly enforces hard physical constraints (workstation type compatibility, machine tooling availability, and operator skill certifications) alongside hard capacity ceilings ($C \leq 100\%$). Rather than employing unconstrained stochastic heuristics that might generate mathematically infeasible schedules under pressure, the system guarantees that every committed reroute plan is physically executable.

21 Strategic Horizons in Autonomous Manufacturing Systems

The implementation of ManufactureFlow establishes an operational foundation for autonomous manufacturing resilience. Future research directions in autonomous in-

dustrial architectures focus on several strategic horizons:

- **Federated Multi-Plant Mesh Coordination:** Extending state-graph coordination across distributed, multi-facility enterprise manufacturing campuses conforming to ISA-95 hierarchical models [1]. In a federated deployment, recovery agents can coordinate inter-facility inventory pooling, evaluate regional freight logistics, and rebalance production schedules across geographically separate manufacturing plants during prolonged workstation outages.
- **Hybrid Diagnostic Topologies:** Investigating advanced diagnostic topologies that fuse continuous high-frequency vibration spectrograms and thermal imaging with deterministic supervisory guardrails. Benchmarking against open datasets such as the UCI AI4I predictive maintenance corpus [8] provides an avenue for exploring the trade-offs between empirical inference accuracy and safety-certified operational bounds.
- **Autonomous Intralogistics & Mobile Robotics Integration:** Interfacing resilience control planes directly with automated warehouse storage systems and Autonomous Mobile Robots (AMRs). Upon automated inventory reservation, the control plane can dispatch an AMR to transport the reserved spare part directly from the warehouse bay to the compromised machining cell, minimizing physical technician transit latency.
- **Dynamic Energy- and Carbon-Aware Scheduling:** Expanding the multi-criteria rerouting objective function to integrate dynamic industrial electricity tariff schedules, peak-demand charges, and machine-specific energy efficiency metrics, enabling production continuity that optimizes both delivery SLAs and factory carbon intensity.

22 Discussion

The fundamental contribution of ManufactureFlow is architectural: **it bridges the chasm between predictive diagnostics and operational execution.** In conventional industry practice, predictive maintenance (PdM) is treated as an end in itself; dashboards display diagnostic alerts, but operational recovery remains fragmented.

ManufactureFlow demonstrates that an event-driven state graph, enforced by deterministic guardrails and ACID relational transactions, can safely automate the administrative and logistical recovery cascade. By executing asset quarantine, stock commitment, vendor requisitions, and production rerouting within milliseconds of digital orchestration, the platform preserves factory capacity and isolates customers from supply chain disruption.

23 Comparative Analysis

Table 5 evaluates ManufactureFlow against conventional industrial software categories as observed in typical fragmented deployments.

24 Business & Operational Implications

While financial ROI must be established through longitudinal enterprise field trials, the architectural model yields direct operational consequences:

- **Reduction of Coordination Latency:** Compresses the execution of digital coordination decisions into sub-second transaction workflows under the tested local environment.
- **Preservation of OEE:** Rerouting active jobs prevents line starvation and maintains factory-wide Overall Equipment Effectiveness (OEE) [2].
- **Mitigation of Freight Premiums:** Early vendor notification eliminates emergency overnight freight charges for stockout spares.
- **Protection of Customer SLA:** Early delivery delta identification allows logistics coordinators to manage customer expectations before contractual delivery penalties apply.

25 Conclusion

ManufactureFlow demonstrates an enterprise-oriented prototype architecture for autonomous industrial operations. By coupling predictive machine diagnostics with transactional state-graph orchestration, the system transforms unexpected equipment degradation from an operational crisis into a structured, self-healing recovery workflow at the operational coordination layer. By enforcing deterministic physical constraints, optimistic relational locking, and human-gated return-to-service validation, the platform establishes a trustworthy bridge between autonomous agentic intelligence and industrial operational reality.

References

- [1] International Society of Automation (ISA), *ISA-95 / IEC 62264: Enterprise-Control System Integration*. The standard defines models and information exchange boundaries between manufacturing operations and enterprise/business functions. <https://www.isa.org/standards-and-publications/isa-standards/isa-95-standard>
- [2] International Organization for Standardization (ISO), *ISO 55000:2024 – Asset management: Vocabulary, overview and principles*, 2nd ed., 2024. <https://www.iso.org/standard/83053.html>
- [3] Occupational Safety and Health Administration (OSHA), *29 CFR 1910.147 – The Control of Hazardous Energy (Lockout/Tagout)*. U.S. Department of Labor. <https://www.osha.gov/laws-regs/regulations/standardnumber/1910/1910.147>
- [4] OPC Foundation, *OPC Unified Architecture – Part 1: Overview and Concepts*. OPC UA provides a platform-independent framework for industrial information exchange between devices, control systems, MES and enterprise systems. <https://reference.opcfoundation.org/specs/OPC-10000-1/4>
- [5] OASIS, *MQTT Version 5.0*, OASIS Standard, 2019. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [6] LangChain, *LangGraph Documentation – Persistence and Durable Execution*. LangGraph provides stateful graph execution, checkpoint persistence, human-in-the-loop interrup-

Table 5: Architectural Comparison with Conventional Industrial Systems (Typical Fragmented Baseline)

Dimension	SCADA / Condition Monitoring	Predictive Maintenance (PdM)	Maintenance	Standard MES / ERP	ManufactureFlow Control Plane
Primary Focus	Sensor threshold alarms	Component failure forecasting		Production recording & ERP sync	Closed-loop operational resilience
Response Mechanism	Passive dashboard indicator	Offline diagnostic report		Manual schedule adjustment	Autonomous cross-functional recovery
Operational Quarantining	None (manual operator intervention)	None		Manual lockout in software	Automated allocation locking (< 480 ms)
Spare Inventory Linking	None	Disconnected lookup	WMS	Manual PO creation	Autonomous reservation & vendor sourcing
Production Continuity	None	None		Manual Gantt reassignment	Automated constraint-matching rerouting
Delivery SLA Impact	None	None		Discovered post-delay	Proactive temporal margin calculation
Auditability	Fragmented alarm historians	Local diagnostic files		Disconnected audit trails	End-to-end lineage under correlation ID
Human Governance	Open-ended manual response	Manual interpretation		Manual workflow approvals	Gated Return-to-Service validation

tion, and fault-tolerant workflow resumption. <https://docs.langchain.com/oss/python/langgraph/persistence>

- [7] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation – Transaction Isolation and Explicit Locking*. PostgreSQL transaction isolation and row-level locking mechanisms provide the database primitives used by the implementation for concurrent resource reservation and state transitions. <https://www.postgresql.org/docs/16/transaction-iso.html>
- [8] UCI Machine Learning Repository, *AI4I 2020 Predictive Maintenance Dataset*, 2020. DOI: 10.24432/C5HS5C. This dataset is identified as a candidate future benchmark for replacing the current controlled failure-prediction provider with a trained predictive-maintenance model. <https://archive.ics.uci.edu/dataset/601/ai4i>

E10 **Stakeholder Notification:** `backend/src/lib/final-stakeholder-notifications/service.ts`. Role-specific notification generation and persistence.

E11 **Return-to-Service Execution:** `backend/src/lib/maintenance-execution/service.ts`. Maintenance lifecycle transitions and human-gated RTS validation.

E12 **Database Schema:** `backend/src/lib/db/schema.ts`. PostgreSQL/Drizzle entity definitions supporting operational state, recovery state and audit records.

E13 **Business Requirements:** `docs/PS.md`. Functional requirements FR-1 through FR-10 and associated operational workflow requirements.

Evidence classification. External references establish the engineering and industrial context of the architecture. Repository evidence establishes implementation claims. Controlled demonstration values and measured benchmark results are reported separately in Sections 8 and 19; they are not treated as external validation.

Implementation Evidence Register

The following repository artifacts provide implementation evidence for the system described in this report. These entries are evidence pointers rather than external academic references.

- E1 **Agent Runtime / State Graph:** `backend/src/lib/agent-graph/graph.ts`. Compiled recovery orchestration graph and node topology.
- E2 **Telemetry and Failure Intelligence:** `backend/src/lib/agents/telemetry-monitor.ts` and `backend/src/lib/agents/failure-prediction.ts`. Telemetry validation, threshold classification and controlled diagnostic-provider integration.
- E3 **Recovery Orchestrator:** `backend/src/lib/agents/recovery-orchestrator.ts`. Workstation risk evaluation, allocation locking and recovery coordination.
- E4 **Resource Recovery:** `backend/src/lib/resource-recovery/service.ts`. Inventory availability checks, reservation logic and procurement branching.
- E5 **Procurement Automation:** `backend/src/lib/procurement-automation/service.ts`. Approved-vendor selection and procurement-request generation.
- E6 **Maintenance Work Orders:** `backend/src/lib/maintenance-work-order/service.ts`. Critical work-order creation and maintenance checklist generation.
- E7 **Recovery Time Estimation:** `backend/src/lib/recovery-time-estimation/service.ts`. Versioned recovery calculation and parameter hashing.
- E8 **Production Rerouting:** `backend/src/lib/production-rerouting/service.ts`. Candidate-machine qualification, capacity validation and job reassignment.
- E9 **Delivery Impact Analysis:** `backend/src/lib/delivery-impact/service.ts`. Shipment commitment and delivery-margin recalculation.